

ComunikApp - Especificação Técnica - Módulo de Expedição e Pós-Produção

Público-alvo: Cursor AI / Desenvolvedor Core

Encoding Mandatório: UTF-8 (Strict) com acentuação correta pt-br

Objetivo: Estruturar a camada que captura a Ordem de Serviço (OS) concluída do Kanban do PCP, permitindo o gerenciamento logístico final, a validação de faturamento e a retroalimentação comercial através da criação de templates baseados no projeto bem-sucedido.

1. Diretriz de Segurança de Dados e Migração Imutável

Para garantir risco zero de perda de dados históricos de clientes e OS antigas durante o deploy:

- **Extensão Não-Invasiva:** Não renomear ou deletar status de banco da tabela `ordem_servicos` que afetem registros passados. Adicionar os novos estados (EXPEDICAO, ENTREGUE, ARQUIVADO) via extensões incrementais do Enum.
- **Nova Entidade Associativa:** As informações de transporte e assinaturas devem ficar numa nova tabela (`expedicoes_logistica`) vinculada à `os_id` através de chave estrangeira única.

2. Modelo de Dados (Prisma Schema)

Adicione os seguintes blocos ao arquivo `backend/prisma/schema.prisma`. Certifique-se de manter o padrão multi-tenant através do campo `loja_id`.

```
// Nota ao dev: Caso StatusOrdemServico seja um enum nativo no MySQL, execute a alteração via
migration manual SQL ALTER TABLE, para não exigir recriação do Enum que corrompe dados.
enum StatusExpedicao {
  AGUARDANDO_SEPARACAO
  PRONTO_PARA_RETIRADA
  EM_ROTA_DE_ENTREGA
  AGUARDANDO_INSTALACAO
```

```

    ENTREGUE_FINALIZADO
    ARQUIVADO
}

enum ModalidadeExpedicao {
    RETIRADA_CLIENTE
    ENTREGA_TRANSPORTADORA
    ENTREGA_FROTA_PROPRIA
    INSTALACAO_NO_LOCAL
}

model ExpedicaoLogistica {
    id          String          @id @default(uuid())
    loja_id     String
    os_id       String          @unique
    modalidade  ModalidadeExpedicao @default(RETIRADA_CLIENTE)
    status      StatusExpedicao   @default(AGUARDANDO_SEPARACAO)

    codigo_rastreio String?      @db.VarChar(100)
    data_expedida   DateTime?
    data_conclusao  DateTime?

    recebedor_nome String?       @db.VarChar(150)
    recebedor_doc   String?       @db.VarChar(50)
    url_assinatura  String?       @db.Text
    observacoes     String?       @db.Text

    criado_em      DateTime       @default(now())
    atualizado_em   DateTime       @updatedAt

    // Relação existente (O Cursor deve conectar corretamente com o modelo de OrdemServico
    // vigente)
    // ordem_servico  OrdemServico  @relation(fields: [os_id], references: [id])

    @@index([loja_id])
    @@map("expedicoes_logistica")
}

```

3. Arquitetura do Backend (NestJS)

Desenvolver o ExpedicaoModule que centralizará a lógica de transição entre produção e entrega.

- **Webhook/Listener do PCP:** O OSPCPIntegrationService deve notificar o ExpedicaoService quando a última etapa produtiva for finalizada. Nesse momento, a OS é retirada da visão produtiva e injetada no fluxo logístico (criando um registro em expedicoes_logistica com status AGUARDANDO_SEPARACAO).
- **Guarda Financeira (Blocker de Expedição):** Antes de permitir a transição para ENTREGUE_FINALIZADO, o backend deve consultar o FinanceiroModule. Se houver faturamento atrelado ao Orçamento da OS pendente (ex: parcela de 50% "Na Entrega" em status "Previsto" ou "Vencido"), a API deve retornar um 409 Conflict (Bloqueio Financeiro), forçando o acerto comercial antes da liberação do material.
- **Ação de Clonagem (Gestão do Conhecimento):** Criar um endpoint POST /expedicao/:os_id/transformar-template. Este serviço varrerá a geometria, insumos, serviços e tempos registrados naquela OS bem-sucedida, persistindo-os numa nova entrada na tabela TemplateProduto, permitindo reuso instantâneo pelos vendedores.

4. Especificação do Frontend (Next.js 15 App Router)

- **Nova Área /expedicao:** Criar um Painel de Expedição (Kanban Logístico ou Tabela Dinâmica), listando unicamente ordens de serviço cujas fases de PCP estão concluídas.
- **Ações do Cartão de Expedição:**
 - Alterar modalidade de envio (ex: Mudar de Retirada para Transportadora).
 - Inserir código de rastreio e atualizar status.
 - **Assinatura Digital de Recebimento:** Implementar um componente de *Canvas* na tela de conclusão de entrega que captura a rubrica do cliente em base64 e salva em url_assinatura, registrando data/hora e o nome completo do recebedor (recebedor_nome).
- **Modal de Bloqueio Financeiro:** Se o backend emitir o erro de parcelas em aberto, disparar um aviso claro vermelho utilizando Tailwind CSS (ex: bg-red-50 text-red-700 p-4 rounded-md), orientando o responsável pela expedição a cobrar o cliente ou acionar o setor financeiro.
- **Visualização "Arquivo Morto":** OSs com status ARQUIVADO perdem acesso de edição. Mostrar apenas em formato "Somente Leitura" para auditoria, oferecendo o botão "Transformar em Novo Template Comercial".